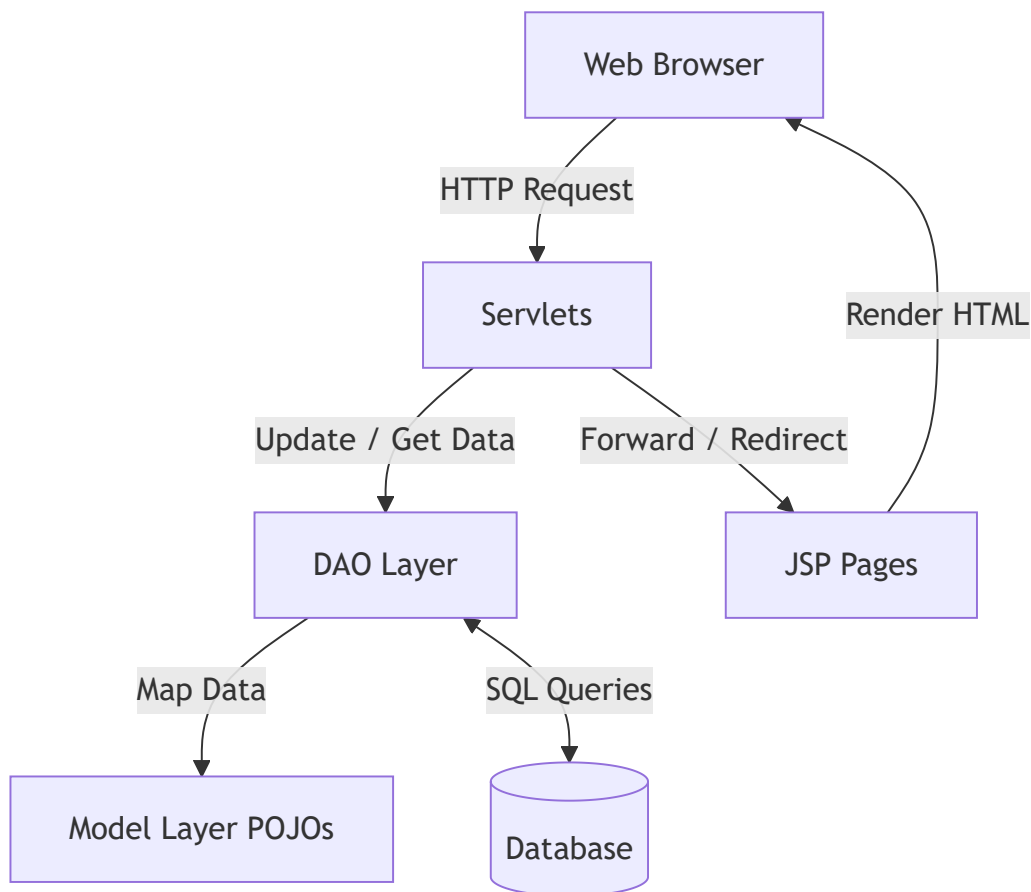


Architecture and Sequence Diagrams

MVC Architecture Overview

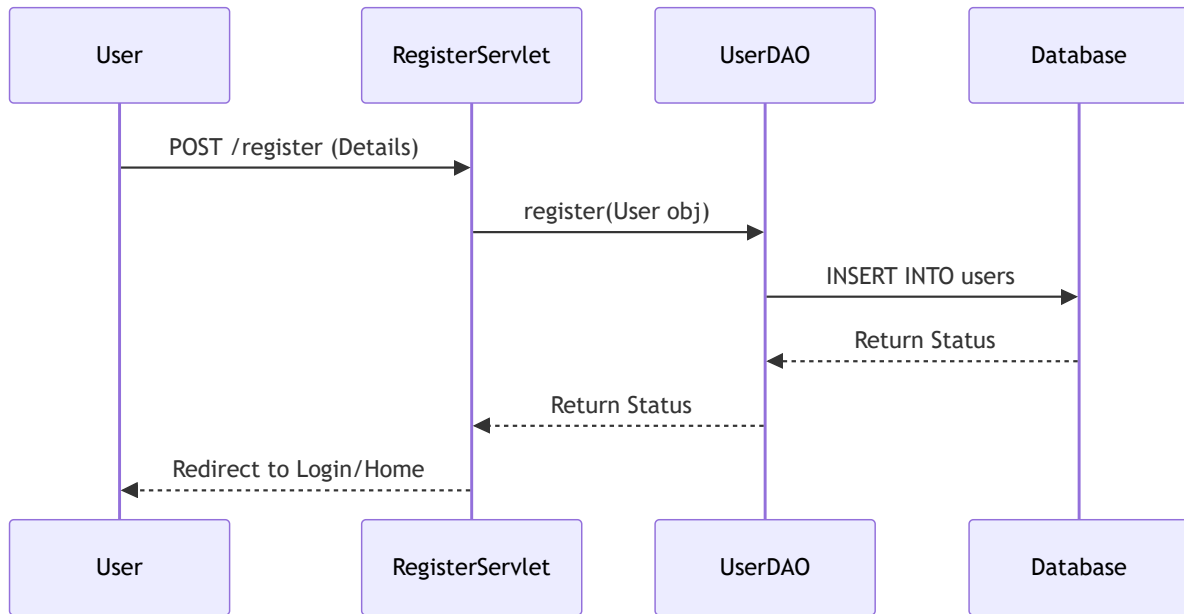
The application follows the Model-View-Controller (MVC) architectural pattern:

- **Model:** Java POJO classes representing the business entities (User, Product, Order, etc.) and Data Access Objects (DAOs) executing the logic to persist and retrieve data from the database.
- **View:** JSP (JavaServer Pages) files handle the presentation logic, displaying data to the users and sending requests to the Controllers. (Inferred based on standard Java EE structure)
- **Controller:** Java Servlets intercept HTTP requests from the views, invoke the appropriate DAOs for data operations, and dispatch the response back to the views.

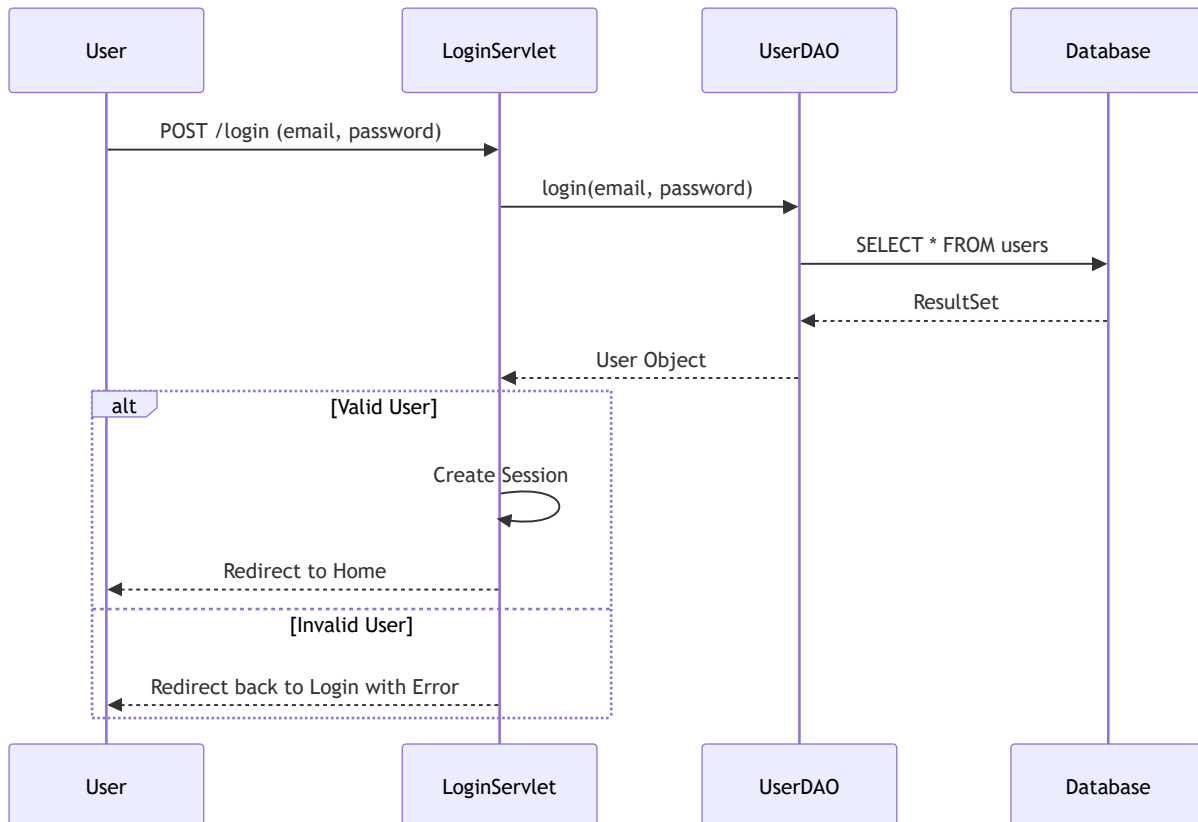


Sequence Diagrams

1. User Registration Flow



2. User Login Flow



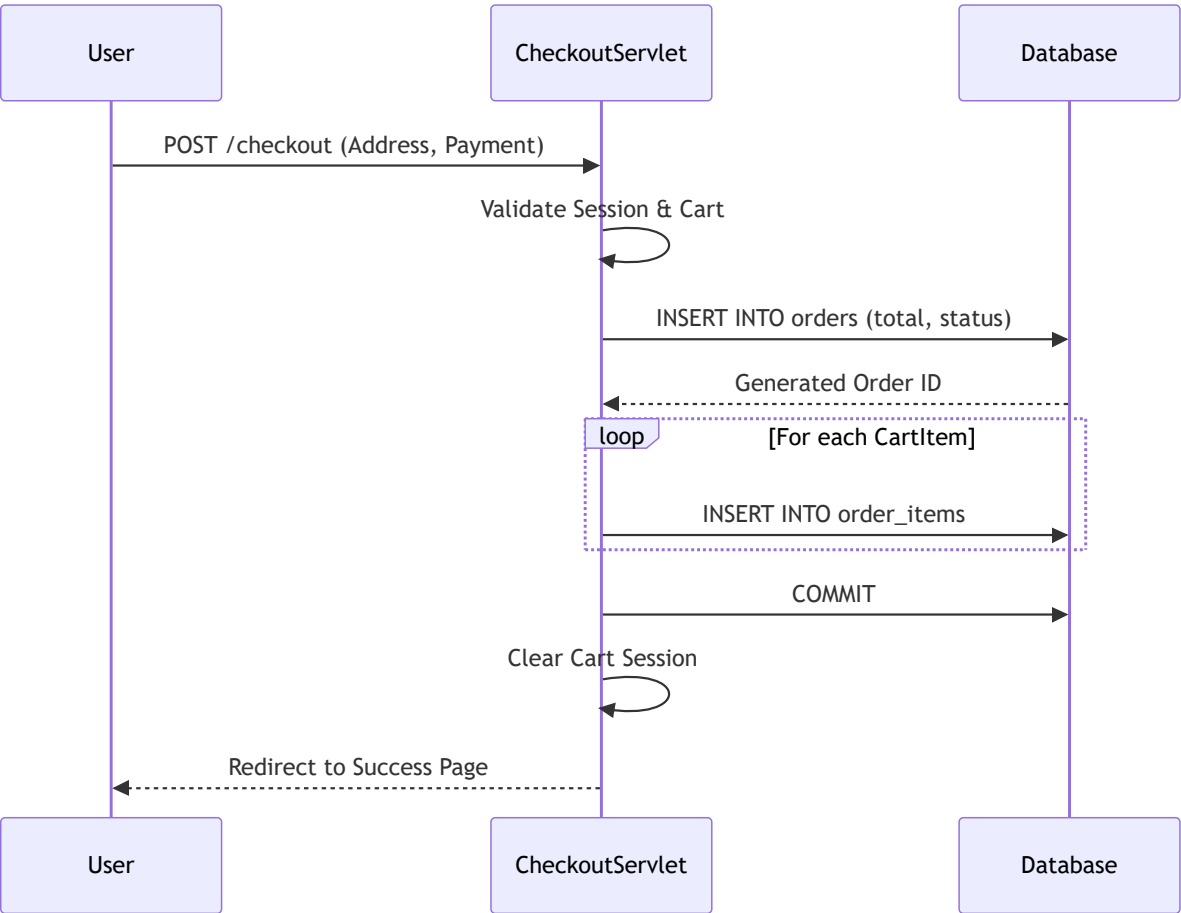
3. Product Browsing Flow



Syntax error in text

mermaid version 11.14.0

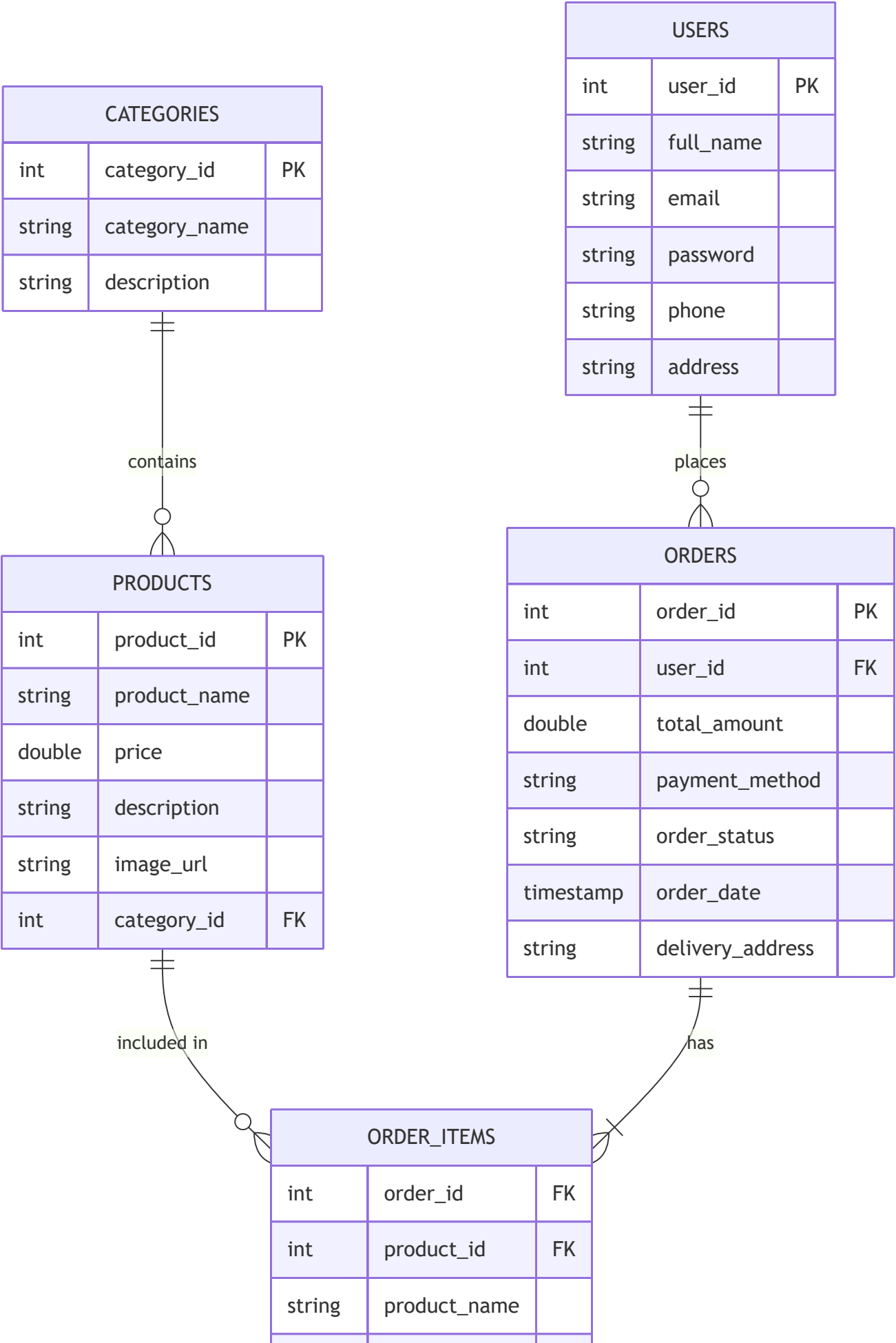
4. Checkout Flow (Direct DB Access Example)



Database Schema and Relationships

This document outlines the database structure, entity relationships, and how the Database tables map to the Java Model classes in the `com.ecommerce.model` package.

Entity Relationship Diagram



int	quantity	
double	unit_price	
double	subtotal	

Models to Table Mapping

The table outlines the relational mapping between the Database Tables and the backend Java classes.

Database Table	Java Model Class	Description
users	User.java	Stores user account information (credentials, contact info).
categories	Category.java	Categorization structure for organizing products.
products	Product.java	Inventory details including names, prices, descriptions, and category associations.
orders	Order.java & OrderGroup.java	Tracks user checkout transactions, timestamps, and order status.
order_items	OrderItem.java & CartItem.java	The junction table detailing which products belong to which order along with snapshot data (price, quantity).

Important Implementation Details

- **CartItem**: Exists largely as a temporary state object during a user session before an order is placed. Once checkout is completed, CartItem data is persisted into the order_items table.
- **OrderGroup**: Used in presentation logic (like viewing order history) to group an Order with a list of its corresponding CartItems.

Controller to DAO Mapping

This document illustrates how each Controller (Servlet) in the application interacts with the underlying Data Access Objects (DAOs) or the database directly to process user requests.

Servlet (Controller)	Target DAO	Method Called	Description of Operation
RegisterServlet	UserDAO	register(User)	Takes user input from registration form and inserts it into the users table.
LoginServlet	UserDAO	login(email, password)	Validates credentials against the users table and initializes a user session.
ProductServlet	ProductDAO	getAllProducts()	Retrieves the full inventory of products from the database for display.
ProductDetailsServlet	ProductDAO	getProductById(id)	Fetches the complete details of a single product for

Servlet (Controller)	Target DAO	Method Called	Description of Operation
			the product detail view.
CategoryServlet	ProductDAO	getProductsByCategory(catId)	Retrieves all products belonging to a specific selected category.
FilterServlet	ProductDAO	filterProducts(category, price)	Builds dynamic SQL queries to search and filter products based on user criteria.
SearchServlet	ProductDAO	searchProducts(keyword)	Performs a LIKE query against product names based on a search string.
CartServlet	<i>None</i>	<i>N/A</i>	Manages the CartItem list entirely within the HttpSession. No direct database interaction.

Servlet (Controller)	Target DAO	Method Called	Description of Operation
RemoveFromCartServlet	<i>None</i>	<i>N/A</i>	Removes an item from the session cart. No database interaction.
CheckoutServlet	<i>Direct DB Connection</i>	<i>Custom SQL</i>	Selects user address, calculates totals, and executes inserts into orders and order_items within a transactional Commit(). Avoids OrderDAO directly.
OrderServlet	<i>Direct DB Connection</i>	<i>Custom SQL</i>	Performs an advanced JOIN operation on orders and order_items tables to map order history data into an OrderGroup mapping.

Data Flow Notes

1. **Transaction Management:** In CheckoutServlet, a direct DB Connection is utilized rather than a DAO in order to enforce standard SQL Transaction safety (`con.setAutoCommit(false)` and `con.commit()`). This guarantees that if inserting `order_items` fails, the orders insertion is rolled back.
 2. **Session State vs Database State:** The "Cart" is managed almost entirely in server memory (HttpSession) using CartItem objects to limit database overhead. It is only flushed to persistent storage (`order_items` table) during the final confirmation phase within CheckoutServlet.
-